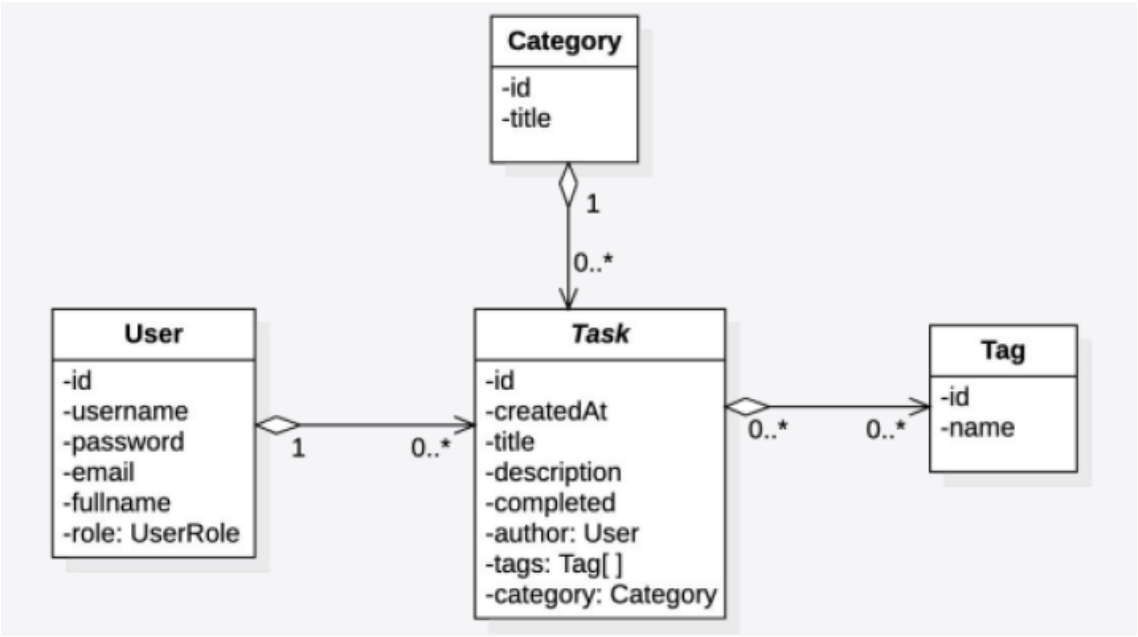
TO DO LIST con API REST

Proyecto Final

Se pretende realizar una aplicación en servidor que permita gestionar una lista de tareas integrando un mecanismo de persistencia de datos en JPA con Hibernate bajo el framework **Spring Boot.** La solución debe permitir la gestión de tareas personales, su categorización y el control de acceso por perfiles.

1. Modelo de datos

El proyecto deberá seguir el modelo de dominio que se muestra en la siguiente figura.



A la entidad **Tarea** se le deberá añadir al menos dos elementos más, a elección propia del alumnado con lo que la aplicación quedará personalizada. La incorporación de un número mayor de características se valorará positivamente.

A continuación, se presenta un listado orientativo de posibles atributos adicionales:

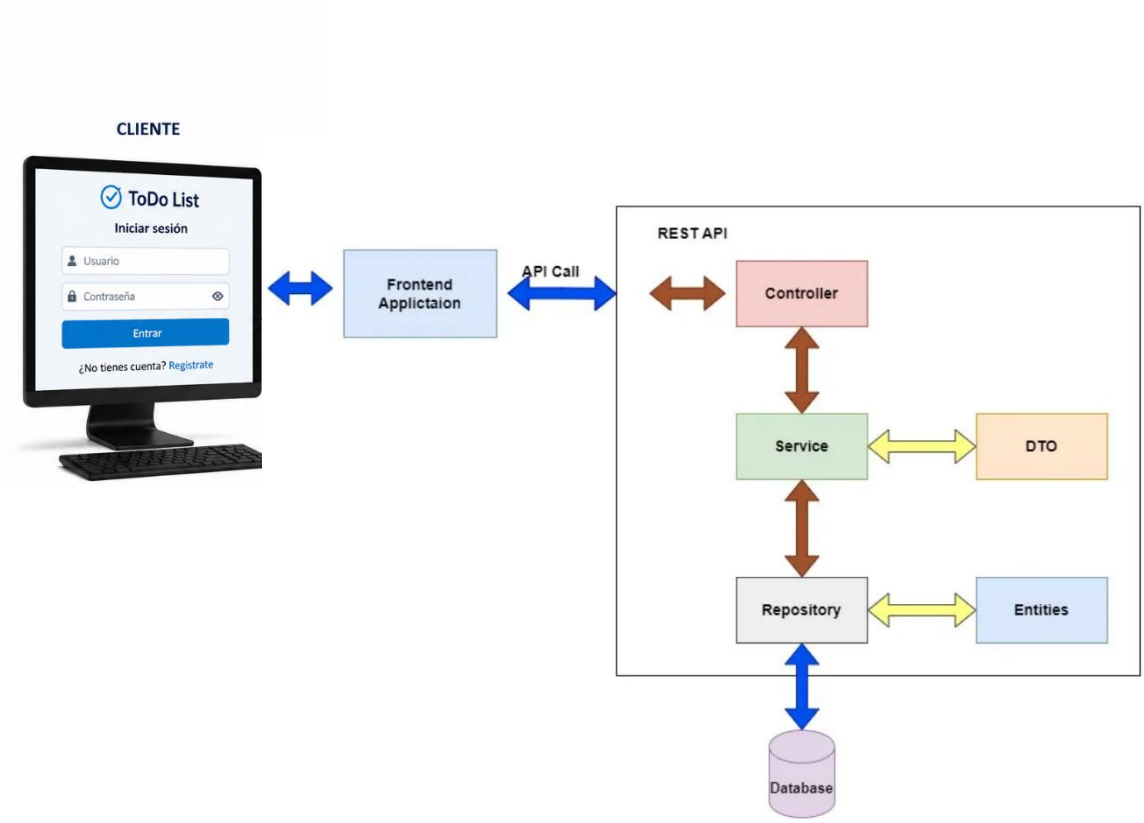
- Fecha límite de finalización (*deadline*)
- Nivel de prioridad de la tarea
- Posibilidad de asignar múltiples categorías a una misma tarea
- Definición de estados de progreso más detallados (por ejemplo: pendiente, en progreso, completada parcialmente, completada)
- Fecha de última actualización de la tarea
- Estimación de tiempo necesario para su realización
- Indicador de importancia o tarea destacada

- Asociación de comentarios o anotaciones adicionales

La selección e implementación de las nuevas características quedará a criterio del alumnado, en función del enfoque adoptado en su desarrollo. Se podrán crear DTOs siempre que se considere conveniente, se deberá justificar su uso. Se **valorará positivamente** la **coherencia** en el diseño del modelo, así como la **justificación** de las decisiones adoptadas en la selección de los atributos incorporados.

2. Arquitectura

Se utilizará una arquitectura MVC similar a la de la siguiente figura.



3. Endpoints

3.1. Admininstrador

- Promocionar usuario a gestor
- Degradar gestor a usuario
- CRUD Usuario
- CRUD categoría

3.2. Gestor

- CRUD categoría

3.3. Usuario

- CRUD Tarea
- CRUD Tag
- Listar categorías

- Eliminar Tags a tareas creadas
- Asignar Tags a tareas creadas
- Buscar tareas con tags seleccionados
- Cambiar contraseña
- Dashboard: Información sobre tareas (por categoría/tag/vencidas/creadas/….)

4. Documentación

La API desarrollada deberá estar documentada utilizando el estándar OpenAPI (Swagger). La documentación deberá:

- Estar disponible mediante una interfaz web accesible desde navegador. **(Obligatorio)**
- Permitir probar los endpoints directamente.
- Incluir descripción de cada operación (GET, POST, PUT, DELETE) **(Obligatorio 20%)**. Definir correctamente:
    - Parámetros de entrada
    - Cuerpos de petición (request body)
    - Respuestas (response)
    - Códigos de estado HTTP

Se valorará positivamente la claridad, completitud y calidad de la documentación generada.

5. Seguridad

La API deberá proporcionar un mecanismo de autenticación y autorización que permita controlar el acceso a los distintos recursos del sistema en función del usuario.

Se utilizará el framework Spring Security como base para la implementación de la seguridad en la aplicación.

El sistema deberá permitir que los usuarios se autentiquen mediante credenciales. La comunicación será Stateless. Se contemplan las siguientes opciones de implementación:

- Autenticación básica mediante usuario y contraseña (Basic Auth)
- Autenticación mediante token (JWT)

El proceso de autenticación deberá:

- Validar las credenciales
- Permitir el acceso a los recursos protegidos únicamente a usuarios autenticados.

Además, el sistema deberá controla el acceso a los recursos en función del rol del usuario. Se definirán los siguientes roles:

- ADMIN, usuario con permisos totales
- GESTOR, usuario que es promocionado por un usuario con rol ADMIN
- USER, usuario estándar que gestiona sus propias tareas

Se deberán proteger los endpoints de la API de acuerdo a lo siguiente:

- Los endpoints públicos (registro, login) estarán accesibles sin autenticación.
- El resto de endpoints deberán requerir autenticación.
- Se deberá configurar Spring Security de forma que la API devuelva códigos HTTP adecuados.

Se establece **obligatorio** en el apartado de seguridad:

- Dependencias de Security
- Clase SecurityConfig configurada

Calificación

Descripción	Puntuación Máxima
Arquitectura y organización	**1**
Modelo de datos y persistencia	**1.5**
Creación del modelo (entidades + relaciones)	0.75
DTOs útiles (justificación)	0.75
End points	**5**
Indicados en el enunciado	3.5
Propios de la ampliación de Tarea	1.5
Documentación – Swagger	**1**
Seguridad	**1.5**

Examen Trimestre miércoles 20 de mayo